# CCSDS 235.1-W-0.4 Reference Implementation — Functional Requirements

Gateway-Based Development Approach

Sponsored by ESA — Organised by AMSAT-UK

April 2026

## Contents


**1 Introduction** **4**
- 1.1 Purpose . . . 4
- 1.2 Scope . . . 4
- 1.3 Reference Documents . . . 4
- 1.4 Definitions and Abbreviations . . . 4

**2 Gateway Overview** **5**

**3 Functional Requirements by Gateway** **6**
- 3.1 Gateway 0 — Design & Architecture . . . 6
  - 3.1.1 FR-0.1: System Architecture . . . 6
  - 3.1.2 FR-0.2: Development Environment . . . 6
  - 3.1.3 FR-0.3: Testing Strategy . . . 6
- 3.2 Gateway 1 — SPDU Layer . . . 6
  - 3.2.1 FR-1.1: Type F1 PLCW Encoding/Decoding . . . 6
  - 3.2.2 FR-1.2: Type F2 PLCW Encoding/Decoding . . . 7
  - 3.2.3 FR-1.3: Variable-Length SPDU Encoding/Decoding . . . 7
  - 3.2.4 FR-1.4: SPDU Validation . . . 7
  - 3.2.5 FR-1.5: PLCW Generation . . . 7
  - 3.2.6 FR-1.6: Directive Decoding . . . 7
  - 3.2.7 FR-1.7: Hex Dump Interoperability . . . 7
- 3.3 Gateway 2 — COP-P Layer . . . 8
  - 3.3.1 FR-2.1: FOP-P Frame Sequencing . . . 8
  - 3.3.2 FR-2.2: FOP-P Transmit Window Management . . . 8
  - 3.3.3 FR-2.3: FOP-P Retransmission . . . 8
  - 3.3.4 FR-2.4: FOP-P PLCW Processing . . . 8
  - 3.3.5 FR-2.5: FARM-P Frame Acceptance . . . 8
  - 3.3.6 FR-2.6: FARM-P PLCW Data Generation . . . 8
  - 3.3.7 FR-2.7: Expedited Service . . . 9
  - 3.3.8 FR-2.8: Sequence Controlled Service . . . 9
  - 3.3.9 FR-2.9: Resynchronization . . . 9
  - 3.3.10 FR-2.10: Persistence Mechanism . . . 9

3.4 Gateway 3 — Frame Layer . . . 9
3.4.1 FR-3.1: P-Frame Transmission . . . 9
3.4.2 FR-3.2: U-Frame Transmission . . . 10
3.4.3 FR-3.3: Frame Reception and Parsing . . . 10
3.4.4 FR-3.4: Frame Version Support . . . 10
3.4.5 FR-3.5: QoS Flag Handling . . . 10
3.4.6 FR-3.6: Frame Format Compliance . . . 10
3.4.7 FR-3.7: Frame Validation . . . 10
3.5 Gateway 4 — Physical Layer Abstraction . . . 11
3.5.1 FR-4.1: Physical Channel Interface . . . 11
3.5.2 FR-4.2: Mock Physical Channel . . . 11
3.5.3 FR-4.3: TCP Physical Channel . . . 11
3.5.4 FR-4.4: UDP Physical Channel . . . 11
3.5.5 FR-4.5: PCID Support . . . 11
3.5.6 FR-4.6: Channel Status Reporting . . . 11
3.6 Gateway 5 — State Machine . . . 11
3.6.1 FR-5.1: State Machine Implementation . . . 12
3.6.2 FR-5.2: State Tables — Full Duplex . . . 12
3.6.3 FR-5.3: State Tables — Half Duplex . . . 12
3.6.4 FR-5.4: State Tables — Simplex . . . 12
3.6.5 FR-5.5: State Transition Validation . . . 12
3.6.6 FR-5.6: State-Controlling Variables . . . 12
3.6.7 FR-5.7: State History Logging . . . 12
3.7 Gateway 6 — Hailing & Session Control . . . 12
3.7.1 FR-6.1: Hailing Mechanism . . . 12
3.7.2 FR-6.2: Caller Role . . . 13
3.7.3 FR-6.3: Responder Role . . . 13
3.7.4 FR-6.4: Session Lifecycle Management . . . 13
3.7.5 FR-6.5: Reconnect (Rehailing) . . . 13
3.7.6 FR-6.6: COMM_CHANGE Support . . . 13
3.7.7 FR-6.7: Session Callbacks . . . 13
3.7.8 FR-6.8: Thread Safety . . . 14
3.8 Gateway 7 — Integration Testing . . . 14
3.8.1 FR-7.1: Session Lifecycle Integration . . . 14
3.8.2 FR-7.2: Reconnect and Resynchronization Integration . . . 14
3.8.3 FR-7.3: Multi-Channel Integration . . . 14
3.8.4 FR-7.4: Performance Benchmarks . . . 14
3.8.5 FR-7.5: Stress Testing . . . 14
3.9 Gateway 8 — Conformance Testing . . . 15
3.9.1 FR-8.1: Mandatory Feature Coverage . . . 15
3.9.2 FR-8.2: State Transition Conformance . . . 15
3.9.3 FR-8.3: SPDU Conformance . . . 15
3.9.4 FR-8.4: COP-P Conformance . . . 15
3.9.5 FR-8.5: Protocol Implementation Conformance Statement (PICS) . . . 15
3.9.6 FR-8.6: Test Vectors . . . 15
3.10 Gateway 9 — Interoperability Testing . . . 15
3.10.1 FR-9.1: SPDU Interoperability . . . 15
3.10.2 FR-9.2: COP-P Interoperability . . . 15

3.10.3 FR-9.3: Session Interoperability . . . 16
3.10.4 FR-9.4: Test Vector Exchange . . . 16
3.10.5 FR-9.5: Workshop Artifacts . . . 16
3.11 Gateway 10 — Documentation & Examples . . . 16
3.11.1 FR-10.1: API Documentation . . . 16
3.11.2 FR-10.2: User Guide . . . 16
3.11.3 FR-10.3: Architecture Documentation . . . 17
3.11.4 FR-10.4: Working Examples . . . 17
3.11.5 FR-10.5: Layer-Specific Documentation . . . 17
3.11.6 FR-10.6: Wire Format Specification . . . 17

**4 Non-Functional Requirements 18**
4.1 NFR-1: Performance . . . 18
4.2 NFR-2: Correctness . . . 18
4.3 NFR-3: Testability . . . 18
4.4 NFR-4: Usability . . . 18
4.5 NFR-5: Maintainability . . . 18
4.6 NFR-6: Portability . . . 18

**5 Technical Constraints 19**
5.1 TC-1: Language and Build . . . 19
5.2 TC-2: Data Formats . . . 19
5.3 TC-3: Dependencies . . . 19
5.4 TC-4: Compatibility . . . 19

**6 Traceability Matrix 20**

**7 Risk Assessment 21**

**8 Acceptance Criteria Summary 22**

**9 Success Metrics 23**

# 1 Introduction

## 1.1 Purpose

This document defines the functional requirements for a reference implementation of CCSDS 235.1-W-0.4 "Space Communications Session Control" (Proximity-1 Session Control). The implementation is structured around a gateway-based development methodology where each gateway delivers a complete, independently testable capability. Contestants are free to choose their programming language, build system, and tooling.

## 1.2 Scope

The implementation covers all software-implementable aspects of the CCSDS 235.1-W-0.4 specification:

- SPDU encoding and decoding (Section 3)
- Persistence mechanism (Section 4)
- Session control state machines (Section 5)
- COP-P protocol — FOP-P and FARM-P (Section 6)
- Frame layer interfaces (Section 5.4–5.6)
- Physical layer abstraction for ground-based testing

RF hardware integration (CCSDS 211.1 Proximity-1 Physical Layer) is out of scope.

## 1.3 Reference Documents

| Document | Description |
|---|---|
| CCSDS 235.1-W-0.4 | Space Communications Session Control specification |
| CCSDS 211.1-B-4 | Proximity-1 Physical Layer (reference only) |
| CCSDS 211.0-B-5 | Proximity-1 Space Link Protocol (frame format) |

The CCSDS 235.1-W-0.4 specification is available from: https://public.ccsds.org/Pubs/235x1w0e4.pdf

## 1.4 Definitions and Abbreviations

| Term | Definition |
|---|---|
| SPDU | Supervisory Protocol Data Unit |
| PLCW | Physical Layer Control Word |
| COP-P | Communications Operation Procedure — Proximity |
| FOP-P | Frame Operation Procedure — Proximity (send side) |
| FARM-P | Frame Acceptance and Reporting Mechanism — Proximity (receive side) |
| PCID | Physical Channel Identifier (0 or 1) |
| QoS | Quality of Service |
| V(S) | Transmitter frame sequence number (FOP-P) |
| V(R) | Receiver frame sequence number (FARM-P) |

## 2 Gateway Overview

This project uses a gateway-based methodology to structure the implementation. A gateway is a development milestone that delivers a complete, independently testable capability. Each gateway delivers tangible outputs, has clear success criteria, can be validated independently, and builds upon previous gateways. Validate thoroughly at each gateway before proceeding — a solid foundation enables faster progress later. See the README for a full explanation.

The project is structured into 11 gateways (0–10), each delivering a testable capability:

| Gateway | Capability | Spec Section | Priority |
|---|---|---|---|
| 0 | Design & Architecture | — | Foundation |
| 1 | SPDU Layer | Section 3 | HIGH |
| 2 | COP-P Layer (FOP-P, FARM-P) | Section 6, Section 4 | HIGH |
| 3 | Frame Layer | Section 5.4–5.6 | HIGH |
| 4 | Physical Layer Abstraction | Section 5.4 | MEDIUM |
| 5 | State Machine | Section 5.1–5.3 | HIGH |
| 6 | Hailing & Session Control | Section 5.1.3, Section 4 | HIGH |
| 7 | Integration Testing | — | HIGH |
| 8 | Conformance Testing | — | HIGH |
| 9 | Interoperability Testing | — | MEDIUM |
| 10 | Documentation & Examples | — | MEDIUM |

# 3 Functional Requirements by Gateway

## 3.1 Gateway 0 — Design & Architecture

**Goal**: Establish system architecture and development foundation.

### 3.1.1 FR-0.1: System Architecture

The system shall define a layered architecture with clear separation between:

- SPDU layer (encoding/decoding)
- COP-P layer (reliable delivery)
- Frame layer (transmission/reception)
- Session layer (state machine, hailing)
- Physical layer (channel abstraction)

### 3.1.2 FR-0.2: Development Environment

The contestant shall select and configure:

- A programming language appropriate for protocol implementation (e.g. C++, Rust, Python, Go, Java, C#)
- A build system appropriate for the chosen language
- A testing framework appropriate for the chosen language
- A documentation generation tool appropriate for the chosen language (e.g. Doxygen, Sphinx, JavaDoc, rustdoc)
- Git for version control

### 3.1.3 FR-0.3: Testing Strategy

The system shall support unit, integration, conformance, performance, and interoperability testing categories.

**Success Criteria**: Architecture documented, build system functional, test framework operational.

## 3.2 Gateway 1 — SPDU Layer

**Goal**: Implement complete SPDU encoding and decoding per CCSDS 235.1-W-0.4 Section 3.

### 3.2.1 FR-1.1: Type F1 PLCW Encoding/Decoding

The system shall encode and decode Type F1 PLCWs (16-bit) containing:

- Report Value (V(R), 8-bit sequence number)
- Expedited Frame Counter (modulo-8)
- PCID (Physical Channel Identifier: 0 or 1)
- Retransmit flag
- Format ID and Type Identifier fields

All multi-byte fields shall use big-endian byte order.

#### 3.2.2 FR-1.2: Type F2 PLCW Encoding/Decoding

The system shall encode and decode Type F2 PLCWs (32-bit) containing:

- Report Value (V(R), 16-bit sequence number)
- Expedited Frame Counter (modulo-8)
- PCID (Physical Channel Identifier: 0 or 1)
- Retransmit flag
- Lockout flag
- Wait flag
- Format ID and Type Identifier fields

All multi-byte fields shall use big-endian byte order.

#### 3.2.3 FR-1.3: Variable-Length SPDU Encoding/Decoding

The system shall encode and decode variable-length SPDUs:

- Type 1: UHF Directives (including SET V(R), SET TRANSMITTER PARAMETERS)
- Type 2: Time Distribution
- Type 3: Status Reports (reserved)
- Type 4: Lunar Gen1 (reserved)
- Type 5: Lunar Gen2 (reserved)

#### 3.2.4 FR-1.4: SPDU Validation

The system shall validate SPDU structure including:

- Format ID correctness
- Type Identifier validity
- Field range constraints
- Minimum/maximum length constraints

Malformed SPDUs shall be rejected with descriptive error information.

#### 3.2.5 FR-1.5: PLCW Generation

The system shall generate PLCWs (Type F1 and F2) from FARM-P state, reflecting current V(R), retransmit flag, lockout flag, wait flag, expedited counter, and PCID.

#### 3.2.6 FR-1.6: Directive Decoding

The system shall decode directives from Type 1 SPDUs and process them to update protocol state, including SET V(R) for resynchronization.

#### 3.2.7 FR-1.7: Hex Dump Interoperability

The system shall produce and consume raw byte arrays (hex dumps) for cross-implementation SPDU exchange.

**Success Criteria**: All SPDU types encode/decode correctly. PLCWs match specification format. Encoding/decoding $< 1$ ms per SPDU. 100% test coverage.

## 3.3 Gateway 2 — COP-P Layer

**Goal**: Implement FOP-P and FARM-P for reliable delivery per CCSDS 235.1-W-0.4 Section 6, with persistence per Section 4.

### 3.3.1 FR-2.1: FOP-P Frame Sequencing

The system shall implement FOP-P (Section 6.2) providing:

- Transmitter sequence number V(S) maintenance
- Support for both 8-bit (modulo-256) and 16-bit (modulo-65536) sequence numbers
- Correct modulo arithmetic per Section 6.1

### 3.3.2 FR-2.2: FOP-P Transmit Window Management

The system shall enforce a transmit window (maximum size 127) that:

- Prevents transmission when the window is full
- Slides the window forward upon PLCW acknowledgment
- Handles window calculations correctly across sequence number wrap-around

### 3.3.3 FR-2.3: FOP-P Retransmission

The system shall retransmit frames when:

- The retransmit flag is set in a received PLCW (R(R)-triggered retransmission)
- A sequence gap is detected (gap-triggered retransmission)
- Progressive retransmission is supported

### 3.3.4 FR-2.4: FOP-P PLCW Processing

The system shall process received PLCWs to:

- Acknowledge transmitted frames (advance NN(R))
- Detect retransmission requests
- Update window state

### 3.3.5 FR-2.5: FARM-P Frame Acceptance

The system shall implement FARM-P (Section 6.3) providing:

- Receiver sequence number V(R) maintenance
- In-sequence frame acceptance (N(S) == V(R))
- Sequence gap detection (N(S) != V(R))
- Duplicate frame detection and discard

### 3.3.6 FR-2.6: FARM-P PLCW Data Generation

The system shall generate PLCW data reflecting:

- Current V(R) (Report Value)
- Retransmit flag (set when gap detected)
- Lockout flag

- Wait flag
- Expedited Frame Counter (modulo-8)

#### 3.3.7 FR-2.7: Expedited Service

The system shall provide Expedited QoS that:

- Bypasses sequence numbering
- Delivers frames immediately without ordering guarantees
- Maintains an independent VE(S) counter
- Increments the Expedited Frame Counter (modulo-8)

#### 3.3.8 FR-2.8: Sequence Controlled Service

The system shall provide Sequence Controlled QoS that:

- Guarantees in-order delivery
- Detects and recovers from frame loss via retransmission
- Uses V(S)/V(R) sequence numbering

#### 3.3.9 FR-2.9: Resynchronization

The system shall support resynchronization (SET V(R) per Section 6.2.3.2) that:

- Resets FARM-P V(R) to a specified value
- Resets FOP-P V(S) correspondingly
- Allows data transfer to continue with new sequence numbers

#### 3.3.10 FR-2.10: Persistence Mechanism

The system shall implement the persistence activity process (Section 4) with configurable:

- WAITING_PERIOD
- RESPONSE timeout
- NOTIFICATION parameters
- LIFETIME parameters

**Success Criteria**: Sequence numbers wrap correctly. Retransmission recovers lost frames. Window management prevents deadlock. Both QoS services operate correctly. 100% test coverage.

### 3.4 Gateway 3 — Frame Layer

**Goal**: Implement frame transmission and reception per CCSDS 235.1-W-0.4 Section 5.

#### 3.4.1 FR-3.1: P-Frame Transmission

The system shall transmit Protocol frames (P-frames) containing SPDUs with:

- Correct PDU Type ID identifying the frame as a P-frame
- SPDU payload (PLCW or directive)
- QoS flags set per specification
- CRC-16 for integrity

#### 3.4.2 FR-3.2: U-Frame Transmission

The system shall transmit User data frames (U-frames) containing application data with:

- Correct PDU Type ID identifying the frame as a U-frame
- User data payload
- Sequence number (for Sequence Controlled service)
- QoS flags set per specification
- CRC-16 for integrity

#### 3.4.3 FR-3.3: Frame Reception and Parsing

The system shall receive frames from the physical layer and:

- Validate frame structure and CRC
- Distinguish P-frames from U-frames via PDU Type ID
- Extract SPDUs from P-frames and route to the SPDU decoder
- Extract user data from U-frames and route to the application

#### 3.4.4 FR-3.4: Frame Version Support

The system shall support:

- Version-3 Transfer Frames
- Version-4 Transfer Frames

#### 3.4.5 FR-3.5: QoS Flag Handling

The system shall correctly set and interpret QoS flags:

- Expedited QoS (bypass sequencing)
- Sequence Controlled QoS (reliable, in-order delivery)

#### 3.4.6 FR-3.6: Frame Format Compliance

Frames shall conform to CCSDS 211.0-B-5 format including:

- Attached Sync Marker (ASM)
- Frame header with version, SCID, VCID fields
- CRC-16 trailer

#### 3.4.7 FR-3.7: Frame Validation

The system shall detect and reject malformed frames including:

- Invalid CRC
- Incorrect frame length
- Invalid version number
- Unrecognized PDU Type ID

**Success Criteria**: P-frames and U-frames distinguished correctly. SPDUs extracted correctly. Frame processing $< 1$ ms per frame. 100% test coverage.

## 3.5 Gateway 4 — Physical Layer Abstraction

**Goal**: Provide physical channel interface and test implementations.

### 3.5.1 FR-4.1: Physical Channel Interface

The system shall define an abstract PhysicalChannel interface supporting:

- `transmit(data)` — send frame data
- `receive()` — receive frame data
- `isConnected()` — query channel status
- PCID identification (Physical Channel ID: 0 or 1)

### 3.5.2 FR-4.2: Mock Physical Channel

The system shall provide a MockPhysicalChannel for unit testing that:

- Operates as an in-memory loopback
- Provides deterministic behavior (no timing dependencies)
- Supports configurable error injection (frame loss, corruption)

### 3.5.3 FR-4.3: TCP Physical Channel

The system shall provide a TCPPhysicalChannel for ground-based testing that:

- Supports client and server modes
- Enables multi-host interoperability testing
- Provides reliable, ordered delivery (TCP semantics)

### 3.5.4 FR-4.4: UDP Physical Channel

The system shall provide a UDPPhysicalChannel for packet loss testing that:

- Supports datagram-based communication
- Enables COP-P retransmission testing under realistic loss conditions
- Supports configurable packet loss rate simulation

### 3.5.5 FR-4.5: PCID Support

The system shall support two physical channels (PCID 0 and PCID 1) operating independently, with PCID correctly reported in PLCWs.

### 3.5.6 FR-4.6: Channel Status Reporting

The system shall report physical channel status including connection state and error conditions.

**Success Criteria**: Mock channel enables deterministic testing. TCP channel enables ground interoperability. UDP channel tests COP-P recovery. PCID support works. 100% test coverage.

## 3.6 Gateway 5 — State Machine

**Goal**: Implement the CCSDS 235.1-W-0.4 state machine per Section 5.

#### 3.6.1 FR-5.1: State Machine Implementation

The system shall implement the session state machine with states:

- Initialization
- Hailing
- Data Services
- Reconnecting
- Termination

#### 3.6.2 FR-5.2: State Tables — Full Duplex

The system shall implement state transition tables for Full Duplex mode per CCSDS 235.1 Tables 5-1 and 5-2, supporting simultaneous bidirectional communication.

#### 3.6.3 FR-5.3: State Tables — Half Duplex

The system shall implement state transition tables for Half Duplex mode per CCSDS 235.1 Tables 5-1 and 5-3, supporting alternating bidirectional communication.

#### 3.6.4 FR-5.4: State Tables — Simplex

The system shall implement state transition tables for Simplex mode per CCSDS 235.1 Tables 5-1 and 5-4, supporting unidirectional communication.

#### 3.6.5 FR-5.5: State Transition Validation

The system shall validate all state transitions and reject invalid transitions with appropriate error reporting.

#### 3.6.6 FR-5.6: State-Controlling Variables

The system shall maintain all state-controlling variables defined in the specification (MODE, ROLE, etc.).

#### 3.6.7 FR-5.7: State History Logging

The system shall log state transitions with timestamps to provide a complete audit trail for debugging and conformance verification.

**Success Criteria**: All transitions match CCSDS 235.1 tables exactly. Invalid transitions rejected. Full/Half/Simplex modes work per specification. 100% test coverage.

### 3.7 Gateway 6 — Hailing & Session Control

**Goal**: Implement link establishment and session management per CCSDS 235.1-W-0.4 Section 5.

#### 3.7.1 FR-6.1: Hailing Mechanism

The system shall implement the hailing procedure per Section 5.1.3 and Tables 5-7, 5-10:

- Persistent transmission of hailing frames by the caller
- Detection of hailing frames by the responder

- Configurable hailing timeout
- Hailing parameter negotiation

#### 3.7.2 FR-6.2: Caller Role

The system shall implement the caller role that:

- Initiates hailing to establish a session
- Transitions through the state machine per specification
- Manages FOP-P (send-side) operations
- Supports reconnect (rehailing) per Tables 5-9, 5-12

#### 3.7.3 FR-6.3: Responder Role

The system shall implement the responder role that:

- Listens for and detects hailing frames
- Responds to hailing within the configured timeout
- Transitions through the state machine per specification
- Manages FARM-P (receive-side) operations

#### 3.7.4 FR-6.4: Session Lifecycle Management

The system shall manage the complete session lifecycle:

- Session establishment (hailing → data services)
- Data transfer phase (P-frames and U-frames)
- Session termination (graceful shutdown)
- Session diagnostics and status reporting

#### 3.7.5 FR-6.5: Reconnect (Rehailing)

The system shall support reconnect per Section 5.1.3 and Tables 5-9, 5-12:

- Detect communication loss (PLCW timeout)
- Initiate rehailing to re-establish the link
- Preserve session state (V(S), V(R) not reset) across reconnect
- Resume data transfer after reconnect

#### 3.7.6 FR-6.6: COMM_CHANGE Support

The system shall support COMM_CHANGE transitions per Section 5.1.3 and Tables 5-8, 5-11 for changing communication parameters during an active session.

#### 3.7.7 FR-6.7: Session Callbacks

The system shall provide a callback interface for application notification of:

- Session established
- Session terminated
- Data received
- Error conditions
- State transitions

#### 3.7.8 FR-6.8: Thread Safety

The session manager shall provide a thread-safe API for concurrent access from multiple application threads.

**Success Criteria**: Hailing establishes link per specification. Reconnect recovers from link loss. COMM_CHANGE works. Session lifecycle completes per specification. 100% test coverage.

### 3.8 Gateway 7 — Integration Testing

**Goal**: End-to-end validation and system testing.

#### 3.8.1 FR-7.1: Session Lifecycle Integration

The system shall pass integration tests covering the complete session lifecycle: hailing → data services → termination, for all duplex modes.

#### 3.8.2 FR-7.2: Reconnect and Resynchronization Integration

The system shall pass integration tests demonstrating:

- Reconnect recovery from simulated communication loss
- Resynchronization (SET V(R)) resetting sequence state
- Data transfer resumption after recovery

#### 3.8.3 FR-7.3: Multi-Channel Integration

The system shall pass integration tests operating on both physical channels (PCID 0 and PCID 1) simultaneously without cross-channel interference.

#### 3.8.4 FR-7.4: Performance Benchmarks

The system shall meet the following performance targets:

| Metric | Target |
|---|---|
| SPDU encoding/decoding | $<$ 1 ms per SPDU |
| Frame processing throughput | $>$ 100 frames/sec |
| Frame processing latency | $<$ 1 ms per frame |
| Session memory overhead | $<$ 50 KB per session |
| Concurrent sessions | 10 or more |

#### 3.8.5 FR-7.5: Stress Testing

The system shall pass stress tests including:

- High frame rate sustained operation
- Multiple concurrent sessions
- Long-duration sessions without resource leaks
- Error recovery under load

**Success Criteria**: All integration tests pass. Performance meets targets. No memory or resource leaks.

## 3.9 Gateway 8 — Conformance Testing

**Goal**: Validate CCSDS 235.1-W-0.4 compliance.

### 3.9.1 FR-8.1: Mandatory Feature Coverage

The system shall implement and test all mandatory features defined in CCSDS 235.1-W-0.4.

### 3.9.2 FR-8.2: State Transition Conformance

The system shall validate that all state transitions match the specification tables exactly, including edge cases and error conditions.

### 3.9.3 FR-8.3: SPDU Conformance

The system shall validate that all SPDU types conform to Section 3, including bit-field layout, byte ordering, and field ranges.

### 3.9.4 FR-8.4: COP-P Conformance

The system shall validate that FOP-P and FARM-P behavior conforms to Section 6, including modulo arithmetic, window management, and retransmission logic.

### 3.9.5 FR-8.5: Protocol Implementation Conformance Statement (PICS)

The system shall produce a PICS document per CCSDS conventions, declaring which features are implemented and their conformance status.

### 3.9.6 FR-8.6: Test Vectors

The system shall generate test vectors in standardized formats (binary + JSON metadata) for cross-implementation validation.

**Success Criteria**: All mandatory features implemented. All conformance tests pass. PICS document complete. 100% conformance.

## 3.10 Gateway 9 — Interoperability Testing

**Goal**: Cross-implementation validation.

### 3.10.1 FR-9.1: SPDU Interoperability

The system shall produce and consume SPDUs that are byte-identical with other conforming implementations for the same input parameters.

### 3.10.2 FR-9.2: COP-P Interoperability

The system shall interoperate with other implementations at the COP-P level:

- FOP-P sending to a remote FARM-P
- FARM-P receiving from a remote FOP-P
- Retransmission and recovery across implementations

#### 3.10.3 FR-9.3: Session Interoperability

The system shall establish sessions with other implementations:

- Caller role connecting to a remote responder
- Responder role accepting a remote caller
- Data transfer across implementations

#### 3.10.4 FR-9.4: Test Vector Exchange

The system shall support the standardized test vector format (Section defined in interoperability.md) for:

- SPDU export/import (binary + JSON metadata)
- Frame export/import (binary + JSON metadata)
- Session event logs (JSON)

#### 3.10.5 FR-9.5: Workshop Artifacts

The system shall produce the 5 interoperability test artifacts required for the mid-project workshop:

| # | Artifact | Specification |
|---|---|---|
| 1 | Type F1 PLCW | Report Value=127, Retransmit=false, PCID=0, Expedited Counter=3 |
| 2 | Type F2 PLCW | Report Value=500, Retransmit=true, PCID=1, Expedited Counter=6 |
| 3 | Variable-Length SPDU | Type 1 Directive, SET V(R) with SEQ_CTRL_FSN=42 |
| 4 | P-frame | Version-3, containing artifact #1, Expedited QoS |
| 5 | U-frame | Version-3, 10-byte payload 0x00–0x09, Sequence Controlled QoS, seq=7 |

**Success Criteria**: Successful session with other implementations. Test vectors decode correctly across implementations. Interoperability report documents compatibility.

### 3.11 Gateway 10 — Documentation & Examples

**Goal**: Complete reference implementation documentation.

#### 3.11.1 FR-10.1: API Documentation

The system shall provide API documentation (using a tool appropriate for the chosen language) covering all public interfaces, classes, methods, and parameters.

#### 3.11.2 FR-10.2: User Guide

The system shall provide a user guide covering:

- Building the project

- Configuration (JSON-based)
- Running examples
- Troubleshooting common issues

#### 3.11.3 FR-10.3: Architecture Documentation

The system shall provide architecture documentation covering:

- Layered design and component interactions
- Data flow through the protocol stack
- Threading model
- Error handling strategy
- Design decisions and rationale

#### 3.11.4 FR-10.4: Working Examples

The system shall provide working example programs:

- Caller station example
- Responder station example
- Interoperability demonstration scripts

#### 3.11.5 FR-10.5: Layer-Specific Documentation

The system shall provide explainer documents for each protocol layer:

- SPDU Layer
- COP-P Layer
- Frame Layer
- Session Layer

#### 3.11.6 FR-10.6: Wire Format Specification

The system shall document the wire format for all protocol data units, enabling other implementations to achieve byte-level compatibility.

**Success Criteria**: All public APIs documented. Examples run successfully. Architecture clearly explained. Build instructions enable anyone to build and run.

# 4 Non-Functional Requirements

## 4.1 NFR-1: Performance

| Metric | Target |
|---|---|
| SPDU encoding/decoding | < 1 ms per SPDU |
| Frame processing | < 1 ms per frame |
| Session memory overhead | < 50 KB per session |
| Concurrent sessions | 10 or more |
| Real-time capability | Process faster than data rate |

## 4.2 NFR-2: Correctness

- 100% conformance with CCSDS 235.1-W-0.4 mandatory features
- All state transitions per specification tables
- Correct modulo-256 and modulo-65536 sequence number arithmetic
- Reliable delivery with Sequence Controlled service (zero data loss)
- Correct PLCW generation reflecting FARM-P state

## 4.3 NFR-3: Testability

- Unit tests for all components (target: 100% coverage per gateway)
- Integration tests for end-to-end flows
- Overall test coverage > 90%
- Reproducible test results (deterministic mock channels)
- Mock implementations for isolated component testing

## 4.4 NFR-4: Usability

- Complete API documentation (using appropriate tool for chosen language)
- Working examples for all major features
- Error messages with actionable guidance
- JSON-based configuration
- Diagnostic and logging support with state history

## 4.5 NFR-5: Maintainability

- Modular, layered architecture with clear separation of concerns
- Consistent coding style following conventions of the chosen language
- Version control with Git
- Continuous integration (recommended)
- Clear component boundaries enabling independent development

## 4.6 NFR-6: Portability

- Cross-platform support (Linux, macOS, or as appropriate for chosen language)
- No RF hardware dependencies for testing
- Standalone operation with minimal external dependencies
- Standard library usage preferred

# 5 Technical Constraints

## 5.1 TC-1: Language and Build

- Implementation language: contestant's choice (e.g. C++, Rust, Python, Go, Java, C#)
- Build system: appropriate for chosen language
- Testing framework: appropriate for chosen language
- Documentation tool: appropriate for chosen language
- Version control: Git

## 5.2 TC-2: Data Formats

- Big-endian byte order for all multi-byte protocol fields
- Binary frame data on the wire
- JSON for configuration files
- Hex dump format for interoperability artifact exchange

## 5.3 TC-3: Dependencies

- Standard library for chosen language
- Threading support (if applicable to chosen language)
- TCP/UDP socket support for physical channel implementations
- Open source dependencies preferred

## 5.4 TC-4: Compatibility

- No RF hardware required for any testing
- TCP and UDP physical channels for ground-based testing only
- Production deployment requires CCSDS 211.1 Proximity-1 Physical Layer integration

# 6 Traceability Matrix

| Functional Requirement | Gateway | Spec Section | Test Category |
|---|---|---|---|
| FR-1.1: Type F1 PLCW | 1 | Section 3 | Unit, Conformance, Interop |
| FR-1.2: Type F2 PLCW | 1 | Section 3 | Unit, Conformance, Interop |
| FR-1.3: Variable-Length SPDUs | 1 | Section 3 | Unit, Conformance |
| FR-1.4: SPDU Validation | 1 | Section 3 | Unit |
| FR-1.5: PLCW Generation | 1 | Section 3, 6.3 | Unit, Integration |
| FR-1.6: Directive Decoding | 1 | Section 3 | Unit, Conformance |
| FR-1.7: Hex Dump Interop | 1 | — | Interop |
| FR-2.1: FOP-P Sequencing | 2 | Section 6.2 | Unit, Conformance |
| FR-2.2: Transmit Window | 2 | Section 6.2, Annex F | Unit, Conformance |
| FR-2.3: Retransmission | 2 | Section 6.2 | Unit, Integration, Interop |
| FR-2.4: PLCW Processing | 2 | Section 6.2 | Unit |
| FR-2.5: FARM-P Acceptance | 2 | Section 6.3 | Unit, Conformance |
| FR-2.6: PLCW Data Generation | 2 | Section 6.3 | Unit, Conformance |
| FR-2.7: Expedited Service | 2 | Section 6 | Unit, Integration |
| FR-2.8: Seq Controlled Service | 2 | Section 6 | Unit, Integration, Interop |
| FR-2.9: Resynchronization | 2 | Section 6.2.3.2 | Unit, Integration |
| FR-2.10: Persistence | 2 | Section 4 | Unit |
| FR-3.1–3.7: Frame Layer | 3 | Section 5.4–5.6 | Unit, Conformance, Interop |
| FR-4.1–4.6: Physical Layer | 4 | Section 5.4 | Unit, Integration |
| FR-5.1–5.7: State Machine | 5 | Section 5.1–5.3 | Unit, Conformance |
| FR-6.1–6.8: Session Control | 6 | Section 5.1.3, Section 4 | Unit, Integration, Interop |
| FR-7.1–7.5: Integration | 7 | — | Integration, Performance |
| FR-8.1–8.6: Conformance | 8 | All | Conformance |
| FR-9.1–9.5: Interoperability | 9 | — | Interop |
| FR-10.1–10.6: Documentation | 10 | — | Review |

# 7 Risk Assessment

| Risk | Impact | Likelihood | Mitigation |
|---|---|---|---|
| State machine complexity | High | Medium | Follow CCSDS tables exactly; extensive unit testing |
| Specification ambiguity (bit-packing, modulo arithmetic) | Medium | High | Cross-team validation at workshop; reference test vectors |
| Threading issues in session manager | High | Medium | Thread-safe design; sanitizers; code review |
| Interoperability failures at workshop | Medium | Medium | Generate comprehensive test vectors; early self-testing |
| Performance bottlenecks | Medium | Low | Profile critical paths; benchmark at each gateway |
| Time constraints | High | Medium | Prioritize HIGH-priority gateways; defer optional features |

# 8 Acceptance Criteria Summary

The implementation shall be considered complete when:

1. All 10 gateways are delivered with passing test suites
2. All functional requirements (FR-1 through FR-10) are implemented
3. All non-functional requirements are met (performance, correctness, testability)
4. PICS document is complete and accurate
5. Interoperability artifacts are generated and validated
6. Documentation enables independent build, configuration, and usage
7. The implementation is ready for cross-team testing at the Goonhilly workshop

# 9 Success Metrics

1. **Completeness**: All 10 gateways delivered
2. **Correctness**: Pass all conformance tests (100% CCSDS 235.1 compliance)
3. **Performance**: Meet all NFR-1 targets
4. **Quality**: Test coverage greater than 90%
5. **Usability**: Complete documentation and working examples
6. **Interoperability**: Successful cross-implementation testing

---

*Document generated for the CCSDS 235.1-W-0.4 Reference Implementation Competition, April 2026. Sponsored by ESA. Organised by AMSAT-UK.*